

# Firebase Login + Synced History

Step-by-step setup guide for ClearScript (HackNEX 2026)

**Repo:** Hacknex-Starter

**Branch:** feat/handwriting-mvp

**Area:** frontend only

**Time:** about 20 minutes

## What this adds

- **Sign in** with Google or email + password, using a button in the top bar. The session survives a page refresh.
- **History sidebar ("task bar")** on the left of the Handwriting page: search, grouped by Today / Yesterday / Earlier, with the open result highlighted. On phones and small screens it opens as the History pop-up.
- **Synced history:** signed in, results are saved to your Firebase account and show up on every device. Signed out, they stay in this browser, as before.
- **Safe to skip:** if the Firebase settings are empty, the app works exactly as before and the sign-in button stays hidden.

## Contents

- **Part 0:** Before you start: read this first
- **Part A:** Firebase console setup (one time)
- **Part B:** Run it on your laptop
- **Part C:** Deploy (Render / Vercel)
- **Part D:** Test checklist
- **Part E:** How the code works
- **Part F:** Troubleshooting
- **Part G:** Optional: backend login check

## Part 0: Before you start (read this first)

### The integration code is already written

Your job is the **Firebase console setup** (Part A) and connecting it with **.env.local** (Part B). Please **don't rebuild** the login or history code: it would duplicate work and cause merge conflicts on the shared branch.

**First check you have it:** after `git pull --rebase` on `feat/handwriting-mvp`, the file `frontend/src/lib/firebase.ts` must exist. If it doesn't, the code hasn't been pushed yet. Ask before starting.

## Setup

- 1 **One Firebase project for the whole team.** Add teammates under **Project settings > Users and permissions**. Never share a Google account password.
- 2 **Stay on the free Spark plan.** Don't add billing or upgrade to Blaze. The free limits (about 20,000 writes and 50,000 reads a day) are far more than we need.
- 3 **Create Firestore in "production mode", not "test mode".** Test mode leaves the database open to anyone and stops working after 30 days. Paste our rules from Part A, step 6 and click **Publish**.
- 4 **Pick region** `asia-south1` (**Mumbai**). It can't be changed later. A wrong region means deleting the project and starting over.

## Secrets and data

- 5 **Never commit** `.env.local`. Git already ignores it. The web `apiKey` isn't really secret, but the **service-account key** from Part G **is**: that one goes only in Render's env vars, never in git, Discord or WhatsApp.
- 6 **Use only sample or fake prescriptions for testing.** Saved results contain prescription text, and real patient data shouldn't end up in a hackathon database.
- 7 **Don't change how history is saved in Firestore.** Each result is stored in one text field called `payload`. That's on purpose: Firestore rejects some of our result data if it's saved as separate fields. If you change where documents are stored ( `users/{uid}/history` ), the security rules must change too.

## Running and testing

- 8 **Restart** `npm run dev` **after editing** `.env.local`. Vite only reads it at start-up. No Sign in button means the values are missing or the server wasn't restarted.
- 9 **Open the app at** `http://localhost:5173`, **not** `127.0.0.1`. Only `localhost` is allowed to sign in by default.
- 10 **Test Google sign-in in normal Chrome or Edge.** Google blocks sign-in inside in-app browsers (WhatsApp or Instagram link previews, embedded webviews). To test on a phone, use the deployed Render URL, not your laptop's IP address.
- 11 **For the live site:** add the Render domain under Authorized domains, set the 4 `VITE_FIREBASE_*` vars on the Render static site, then **Clear build cache & deploy**. The values only take effect after a rebuild.
- 12 **Run the privacy check** (Part D): sign in with a second account and confirm it can't see the first account's history. That shows the rules are working.

### Demo-day backup

If Firebase misbehaves on the day, empty the 4 `VITE_FIREBASE_*` values and rebuild. The app then works exactly as before, with history saved in the browser. Also sign in on the demo laptop **before** judging starts.

## Part A: Firebase console setup (one time, about 10 min)

Only **one person** on the team needs to do this. Then share the 4 config values from step 2 with the team.

- 1 **Create the project.** Go to `console.firebase.google.com`, then **Add project**, and name it e.g. `clearscript`. Turn **Google Analytics off**: we handle prescriptions, so we collect as little data as possible.
- 2 **Register a web app and copy its config.** Click the gear icon, then **Project settings**. Under *Your apps*, click the `</>` (Web) icon, name it `clearscript-web`, and leave "Firebase Hosting" **unticked**. Firebase shows a `firebaseConfig` object. You need 4 of its values:  

```
apiKey, authDomain, projectId, appId
```
- 3 **Turn on the login methods.** Open **Build > Authentication > Get started > Sign-in method** and enable:
  - **Google**: pick a support email and save.
  - **Email/Password**: turn on the first switch only (not "Email link") and save.
- 4 **Authorize your domains.** In **Authentication > Settings > Authorized domains**, `localhost` is already listed. Add the hosted frontend domain too, e.g. `clearscript-hacknex.onrender.com` (and any Vercel URL you use). *If you skip this step, the Google login popup fails on the live site.*

- 5 **Create the database.** Open **Build > Firestore Database > Create database**. Choose **Standard edition** if asked, location `asia-south1` (**Mumbai**), the closest to Coimbatore, then **Start in production mode**. The location can't be changed later.
- 6 **Paste the security rules.** Open **Firestore > Rules**, replace everything with the rules below, and click **Publish**. They let each signed-in user read and write *only their own* data:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{uid}/{document=**} {
      allow read, write: if request.auth != null && request.auth.uid == uid;
    }
  }
}
```

### Is the apiKey a secret?

No. A Firebase *web* apiKey only identifies the project. Your data is protected by the security rules in step 6, not by hiding the key. We still keep it out of git (in `.env.local`), so each environment can point at its own project.

## Part B: Run it on your laptop

- 1 **Get the latest code.** In PowerShell, from the repo folder:

```
git checkout feat/handwriting-mvp
git pull --rebase
```
- 2 **Install packages.** This also installs the new `firebase` package:

```
cd frontend
npm install
```
- 3 **Create** `frontend/.env.local`. Copy `frontend/.env.example` and fill in the 4 values from Part A, step 2:

```
VITE_FIREBASE_API_KEY=AIza...your key...
VITE_FIREBASE_AUTH_DOMAIN=clearscript-xxxxx.firebaseio.com
VITE_FIREBASE_PROJECT_ID=clearscript-xxxxx
VITE_FIREBASE_APP_ID=1:1234567890:web:abc123...
```

`.env.local` is ignored by git, so it is never committed. Don't put quotes around the values.
- 4 **Start the app**, either the usual way ( `.\dev.ps1` from the repo root) or in two terminals:

```
cd backend; uvicorn app.main:app --reload --port 8000
cd frontend; npm run dev
```

Open `http://localhost:5173`. If the dev server was already running, **restart it**: Vite only reads `.env.local` at start-up.
- 5 **Check it.** A **Sign in** button now appears in the top bar, next to the theme switch. On a wide screen (1280 px or more) the **History** sidebar is docked on the left of the Handwriting page.

## Part C: Deploy

---

The Firebase values are **baked in when the site is built** (like `VITE_API_URL`), so they must be set on the host and the site rebuilt.

### Render (our `render.yaml` blueprint)

- 1 Open the Render dashboard, select the static site **clearscript-hacknex**, then **Environment**. The blueprint already lists the 4 keys ( `VITE_FIREBASE_API_KEY` , `..._AUTH_DOMAIN` , `..._PROJECT_ID` , `..._APP_ID` ). Fill in the values.
- 2 Click **Manual Deploy > Clear build cache & deploy**.
- 3 Check that the site's domain is in Firebase **Authorized domains** (Part A, step 4).

### Vercel (if you use it instead)

Go to **Project > Settings > Environment Variables**, add the same 4 keys, **redeploy**, and add the `*.vercel.app` domain to Authorized domains.

#### Nothing to change on the backend

Login and history live entirely in the browser and Firebase. The FastAPI backend and its env vars stay the same.

## Part D: Test checklist

---

- [ ] **Signed out:** digitize a page. It appears in the History sidebar with "Saved in this browser only."
- [ ] Click **Sign in > Continue with Google**. The avatar appears and a "Signed in as ..." toast shows.
- [ ] The sidebar offers "**Move to my account**" for the browser-only results. Click it; they move to the account.
- [ ] Digitize a page while signed in. In the Firebase console, **Firestore > Data** shows `users/<uid>/history/<id>`.
- [ ] Correct a flagged word, then refresh the page. The correction is still there when you reopen it from History.
- [ ] Sign in with the **same account in another browser or phone**. The same history shows up, and new results appear live.
- [ ] **Email sign-up:** "Create an account", then sign out and sign back in. A wrong password shows a friendly error.
- [ ] **Forgot password** sends a reset email (check spam).
- [ ] Sign out. The screen clears and history shows only what this browser has.
- [ ] **Privacy check:** sign in with a *different* account. You must **not** see the first account's results.
- [ ] **Phone width:** the History button opens the list as a pop-up, and the sign-in button shows as an icon.

## Part E: How the code works

### 1. Sign in

AccountMenu.tsx opens the Firebase Auth Google popup or email form

→

### 2. Who is signed in

auth.tsx (AuthProvider) shares the user with the whole app

→

### 3. Where history lives

Handwriting.tsx: signed in, Firestore; signed out, browser localStorage

→

### 4. Shown in

HistoryPanel.tsx: sidebar on wide screens, pop-up otherwise

| File                                         | What it does                                                                                                                                                                                    |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| src/lib/firebase.ts (new)                    | Starts Firebase from the <code>VITE_FIREBASE_*</code> values. Exports <code>firebaseEnabled</code> (false = no config, so sign-in is hidden), <code>auth</code> and <code>db</code> .           |
| src/lib/auth.tsx (new)                       | <code>AuthProvider</code> + <code>useAuth()</code> : user, <code>signInGoogle</code> , <code>signInEmail</code> , <code>signUpEmail</code> , <code>resetPassword</code> , <code>logout</code> . |
| src/lib/cloudHistory.ts (new)                | Reads and writes history in Firestore: <code>watchHistory</code> (live list, newest 100), <code>saveCloudEntry</code> , <code>batchCloud</code> (bulk move or delete).                          |
| src/components/AccountMenu.tsx (new)         | Top-bar Sign in button / avatar, the sign-in dialog (Google + email, sign-up, forgot password) and friendly error messages.                                                                     |
| src/pages/clearscript/HistoryPanel.tsx (new) | <code>HistoryList</code> (search + day groups), <code>HistorySidebar</code> (the docked task bar), <code>HistoryStatus</code> (where it's saved + "Move to my account").                        |
| src/pages/Handwriting.tsx                    | Picks the history source based on sign-in, saves new results and corrections, docks and hides the sidebar (remembered per browser).                                                             |
| src/pages/clearscript/Dialogs.tsx            | The History pop-up now reuses <code>HistoryList</code> , and "Delete all" works for both storage types.                                                                                         |
| src/App.tsx                                  | Wraps the app in <code>AuthProvider</code> , adds <code>AccountMenu</code> to the header, and updates the privacy text.                                                                         |
| src/i18n/en/common.ts ,<br>clearscript.ts    | All new English text ( <code>auth.*</code> , <code>cs.history.*</code> ). Other languages fall back to English.                                                                                 |
| .gitignore , .env.example ,<br>render.yaml   | Ignore <code>.env.local</code> , document the 4 keys, and add them to the Render static site.                                                                                                   |

## What is stored in Firestore

One document per result at `users/{uid}/history/{entryId}` :

|                  |                                                                          |
|------------------|--------------------------------------------------------------------------|
| id, ts, updated  | id and times (ms)                                                        |
| name, thumb      | file name and a tiny 96 px JPEG preview (the full image is never stored) |
| docType, flagged | "prescription"   "note", and how many words still need checking          |
| payload          | JSON string of { result, words } including the user's corrections        |

The result is saved as one JSON string because Firestore rejects some shapes our results can have (empty keys inside `votes`, `undefined` values). A string comes back exactly as it was saved. Each document must stay under 1 MB; a normal page is well below that.

## Part F: Troubleshooting

| You see                                                               | Fix                                                                                                                                                                                                                           |
|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| No <b>Sign in</b> button at all                                       | <code>.env.local</code> is missing, has an empty value, or is in the wrong folder (it must be in <code>frontend/</code> ). Restart <code>npm run dev</code> after editing it. On Render or Vercel: set the vars and redeploy. |
| "This site isn't allowed to sign in yet"                              | Add the domain in <b>Authentication &gt; Settings &gt; Authorized domains</b> .                                                                                                                                               |
| "This sign-in method is turned off"                                   | Enable Google or Email/Password in <b>Authentication &gt; Sign-in method</b> .                                                                                                                                                |
| "Your browser blocked the Google window"                              | Allow pop-ups for the site, then try again.                                                                                                                                                                                   |
| "Couldn't load your saved history" or "Couldn't save to your account" | The Firestore database isn't created yet, or the rules weren't published (Part A, steps 5–6). Also check your internet connection.                                                                                            |
| Console error <code>auth/api-key-not-valid</code>                     | The apiKey was copied wrong, or has quotes or spaces. Copy it again from Project settings.                                                                                                                                    |
| History sidebar not visible                                           | It docks only on screens 1280 px or wider. Click <b>History (n)</b> to show it again if it was hidden. On smaller screens it opens as a pop-up.                                                                               |
| Sign-in works locally but not on the live site                        | The env vars weren't set before the build, or the domain isn't authorized. Set both and redeploy.                                                                                                                             |

## Part G (optional): let the backend know who is signed in

This is **not needed** for login or history. Use it only if the backend should also tie data to the user, for example storing writer-profile corrections per account instead of per writer name.

- 1 **Frontend:** send the user's ID token with API calls:

```
const token = await auth.currentUser?.getIdToken();
headers: { Authorization: `Bearer ${token}` }
```

- 2 **Backend:** `pip install firebase-admin`. In Firebase, open **Project settings > Service accounts > Generate new private key**. Store that JSON as a **Render env var** (never in git), then check the token in FastAPI:

```
from firebase_admin import auth as fb_auth, credentials, initialize_app
initialize_app(credentials.Certificate(json.loads(os.environ["FIREBASE_SERVICE_ACCOUNT"])))

def current_uid(request: Request) -> str:
    token = request.headers.get("authorization", "").removeprefix("Bearer ")
    return fb_auth.verify_id_token(token)["uid"] # raises if invalid or expired
```

### Reminder for the demo

ClearScript is a reading aid, not a medical device. Saved results contain prescription text, so the security rules in Part A, step 6 are what keep each account's data private. Don't loosen them for testing.